

Summary of Backpropagation-Free Training

Geng Tian

March 24, 2025

Last week, I conducted a series of experiments to evaluate the effectiveness of the Unified Likelihood Ratio (ULR) method for gradient estimation. Specifically, I trained 2-Layer CNN on the MNIST dataset and ResNet-18 on the CIFAR-10 dataset for classification tasks. In the following, I describe the experimental setup, present the results, and discuss key findings.

1 Gradient Estimation in ULR

The ULR method estimates gradients based on the model parameters θ and the data (x, y) as follows:

$$\nabla_{\theta} \mathbb{E}[L] = \mathbb{E} \left[\frac{1}{\sigma} L(v, y) \epsilon \right], \quad (1)$$

where L denotes the loss function, and $v = f(x; \theta + \sigma \epsilon)$ represents the model output with added noise $\sigma \epsilon$. Here, σ is a hyperparameter controlling the scale of the perturbation, and ϵ is sampled from a standard normal distribution $\mathcal{N}(0, 1)$.

The method employs a symmetric perturbation strategy to reduce the gradient estimation variance. Specifically, it computes the gradients using both positive and negative perturbations simultaneously and averages the results. The final gradient estimate is given by:

$$\nabla_{\theta} L \propto \frac{1}{2\sigma} (L(f(x; \theta + \sigma \epsilon), y) - L(f(x; \theta - \sigma \epsilon), y)) \cdot \epsilon. \quad (2)$$

2 Experimental Setup

The hyperparameters are carefully tailored to match the unique characteristics of each dataset. To ensure stability in gradient estimation, the gradient is computed 10 times and averaged in each iteration.

For training MNIST, the Adam optimizer is utilized with a learning rate of 0.001. The noise scale (σ) is set to 0.01, and the batch size is configured to 64. Training is conducted for 10 epochs. For training CIFAR-10, the SGD optimizer is employed with a learning rate of 0.01. The noise scale is reduced to 0.001, and the batch size is increased to 128. The training process is extended to 200 epochs to accommodate the increased complexity of this dataset.

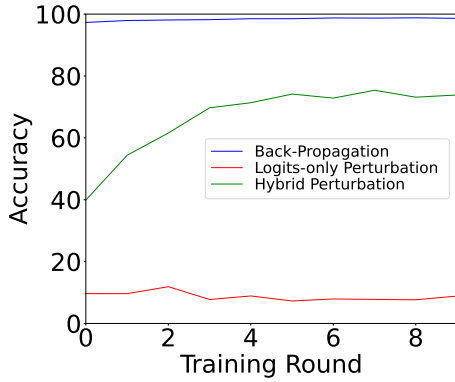
3 Experimental Results

I compare two back-propagation-free perturbation strategies : logits-only perturbation, where the perturbation is added only to the logits, and hybrid perturbation, where noise is added to both model parameters and logits. Additionally, I contrast these back-propagation-free methods with a back-propagation method to evaluate their relative performance. Table 1 summarizes the accuracies for these strategies. The results demonstrate that the ULR method is effective for training CNNs on MNIST but fails to achieve comparable performance when training ResNet18 on CIFAR-10. Furthermore, the accuracy achieved by hybrid perturbation is consistently higher than that of logits-only perturbation.

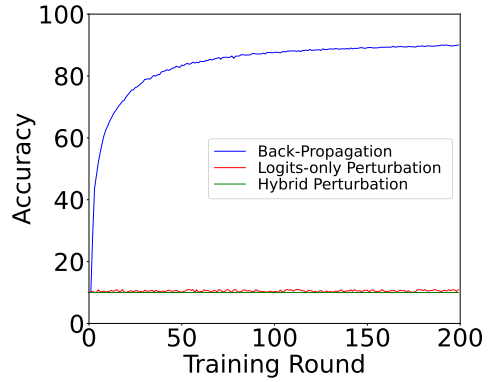
Figures 1a and 1b illustrate the test accuracy curves over epochs for MNIST and CIFAR-10, respectively. These figures provide insights into the convergence behavior of each perturbation strategy.

Table 1: Classification accuracies for different perturbation strategies.

Dataset	Perturbation Strategy	Test Accuracy (%)
MNIST	Back-Propagation	98.63
	Logits-only Perturbation	8.85
	Hybrid Perturbation	73.86
CIFAR-10	Back-Propagation	90.02
	Logits-only Perturbation	10.00
	Hybrid Perturbation	10.00



(a) MNIST



(b) CIFAR-10

Figure 1: Accuracy under different gradient calculation methods

4 Discussion

In my experiments, I did not utilize the VGG8 and ResNet5 models proposed in [1]. This decision was motivated by two key factors: First, these models have not been used in other work. Second, [1] does not provide detailed parameter configurations for them. The only information that can be inferred is that these two models are lightweight versions modified from VGG and ResNet, respectively.

My experimental results demonstrate that the ULR method is effective for lightweight 2-Layer CNN models but performs poorly on larger architectures such as ResNet18. These findings align with the results reported in [1]. The underlying reason is that the ULR method only estimates a single gradient value for all layers, which are shared across the whole network.

Inspired by this analysis, I propose a hybrid approach for split learning. In this framework, the server utilizes backpropagation to compute and update gradients, leveraging its capacity for large-scale computations. Meanwhile, clients employ the ULR method to estimate gradients locally. By combining these two techniques, the model can maintain overall accuracy while reducing the computational burden on resource-constrained clients.

References

- [1] J. Jiang, Z. Zhang, C. Xu, Z. Yu, and Y. Peng, “One forward is enough for neural network training via likelihood ratio method,” *arXiv preprint arXiv:2305.08960*, 2023.